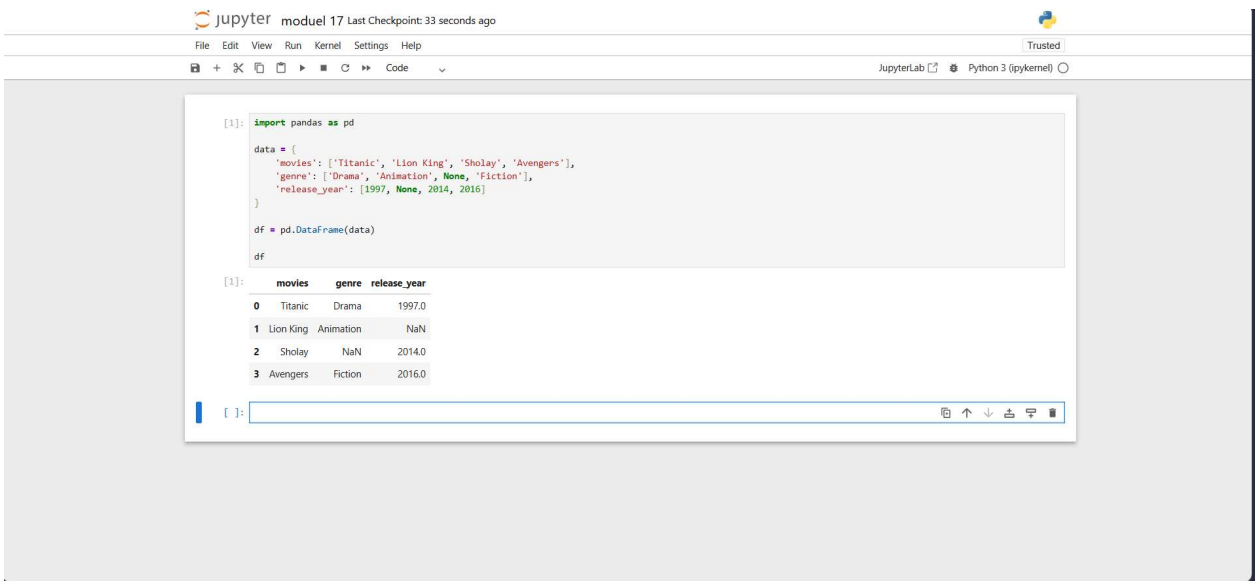
Data Wrangling

Sometimes, data Wrangling is referred to as **data munging**. It is the process of transforming and mapping data from one "raw" data form into another format to make it more appropriate and valuable analytics.

• Consider an example of data wrangling with a focus on **handling missing values** and **creating new features.**

• Suppose you have a dataset containing information about movies, but it has missing values!

```
data = {
    'Title': ['Movie1', 'Movie2', 'Movie3', 'Movie4', 'Movie5'],
    'Genre': ['Action', 'Comedy', 'Drama', None, 'Action'],
    'ReleaseYear': [2010, 2012, None, 2015, 2018],
    'BoxOffice': [50000000, 80000000, 30000000, None, 120000000]
}
```



jupyter moduel 17 Last Checkpoint: 2 minutes ago

File Edit View Run Kernel Settings Help Trusted

Code JupyterLab Python 3 (ipykernel)

0	Titanic	Drama	1997.0
1	Lion King	Animation	NaN
2	Sholay	NaN	2014.0
3	Avengers	Fiction	2016.0

[2]:
```
import pandas as pd

data = {
    'movies': ['Titanic', 'Lion King', 'Sholay', 'Avengers'],
    'genre': ['Drama', 'Animation', None, 'Fiction'],
    'release_year': [2012, None, 2014, 2016],
    'boxoffice': [500000, 520000, None, 1000000]
}

df = pd.DataFrame(data)

df
```

[2]:

	movies	genre	release_year	boxoffice
0	Titanic	Drama	2012.0	500000.0
1	Lion King	Animation	NaN	520000.0
2	Sholay	NaN	2014.0	NaN
3	Avengers	Fiction	2016.0	1000000.0

[ ]:

jupyter moduel 17 Last Checkpoint: 2 minutes ago

File Edit View Run Kernel Settings Help Trusted

Code JupyterLab Python 3 (ipykernel)

[3]:
```
import pandas as pd

data = {
    'movies': ['Titanic', 'Lion King', 'Sholay', 'Avengers'],
    'genre': ['Drama', 'Animation', None, 'Fiction'],
    'release_year': [2012, None, 2014, 2016],
    'boxoffice': [500000, 520000, None, 1000000]
}

df = pd.DataFrame(data)

df['genre'].fillna('Unknown', inplace=True)

df
```

```
C:\Users\jithi\AppData\Local\Temp\ipykernel_17616\1604380567.py:12: ChainedAssignmentError: A value is being set on a copy of a DataFrame or Series thr
ough chained assignment using an inplace method.
Such inplace method never works to update the original DataFrame or Series, because the intermediate object on which we are setting values always behav
es as a copy (due to Copy-on-Write).

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' instead, to perform the operation inpl
ace on the original object, or try to avoid an inplace operation using 'df[col] = df[col].method(value)'.

See the documentation for a more detailed explanation: https://pandas.pydata.org/pandas-docs/stable/user_guide/copy_on_write.html
  df['genre'].fillna('Unknown', inplace=True)
```

[3]:

	movies	genre	release_year	boxoffice
0	Titanic	Drama	2012.0	500000.0
1	Lion King	Animation	NaN	520000.0
2	Sholay	NaN	2014.0	NaN
3	Avengers	Fiction	2016.0	1000000.0

[ ]:

Filling missing values with median:-

jupyter moduel 17 Last Checkpoint: 6 minutes ago

File Edit View Run Kernel Settings Help Trusted

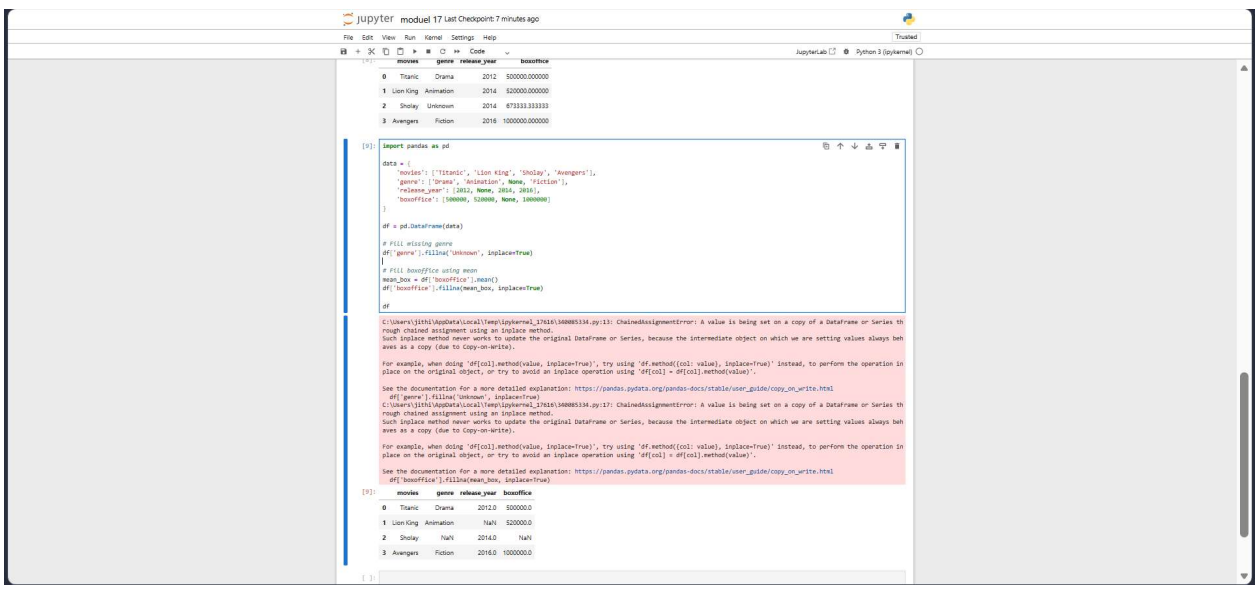
Code JupyterLab Python 3 (ipykernel)

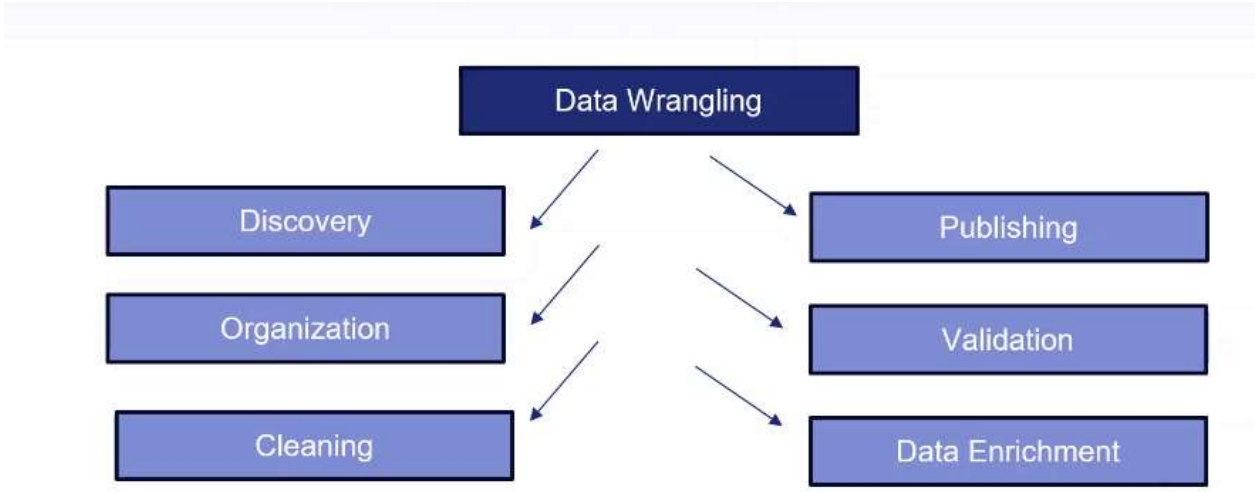
[8]:
```python
import pandas as pd

data = {
    'movies': ['Titanic', 'Lion King', 'Sholay', 'Avengers'],
    'genre': ['Drama', 'Animation', None, 'Fiction'],
    'release_year': [2012, None, 2014, 2016],
    'boxoffice': [500000, 520000, None, 1000000]
}

df = pd.DataFrame(data)

df['genre'] = df['genre'].fillna('Unknown')

median_year = df['release_year'].median()
df['release_year'] = df['release_year'].fillna(median_year)


df['release_year'] = df['release_year'].astype(int)


mean_box = df['boxoffice'].mean()
df['boxoffice'] = df['boxoffice'].fillna(mean_box)

df
```

[8]:

	movies	genre	release_year	boxoffice
0	Titanic	Drama	2012	500000.000000
1	Lion King	Animation	2014	520000.000000
2	Sholay	Unknown	2014	673333.333333
3	Avengers	Fiction	2016	1000000.000000

Filling missing values with mean:-





Discovery: Before starting the wrangling process, it is critical to think about what may lie beneath your data. It is crucial to think critically about what results you anticipate from your data and what you will use it for once the wrangling process is complete. Once you've determined your objectives, you can gather your data.

Organization: After you've gathered your raw data within a particular dataset, you must structure your data. Due to the variety and complexity of data types and sources, raw data is often overwhelming at first glance.

Source

- text file
- excel file
- CSV
- html files (web scrapping)
- images data

Cleaning involves removing outliers, formatting nulls, and eliminating duplicate data. It is important to note that cleaning data collected from web scraping methods might be more tedious than cleaning data collected from a database. Essentially, web data can be highly unstructured and require more time than structured data from a database.

Data enrichment: This step requires that you take a step back from your data to determine if you have enough data to proceed. Finishing the wrangling process without enough data may compromise insights gathered from further analysis. For example, investors looking to analyze product review data will want a significant amount of data to portray the market and increase investment intelligence.

Validation: After determining you gathered enough data, you will need to apply validation rules to your data. Validation rules, performed in repetitive sequences, confirm that your data is

consistent throughout your dataset. Validation rules will also ensure quality as well as security. This step follows similar logic utilized in data normalization, a data standardization process involving validation rules.

Publishing: The final step of the data munging process is data publishing. Data publishing involves preparing the data for future use. This may include providing notes and documentation of your wrangling process and creating access for other users and applications.

Scikit -learn:-

• Open-source machine learning library

• Efficient tools for data analysis and modeling

• Includes various machine learning algorithms

• Scikit-learn provides a consistent and easy-to-use API for various machine learning tasks, making it accessible for both beginners and experienced practitioners

Let's consider a simple use case: predicting whether a student will **pass or fail** an exam based on the **number of hours** they studied. This is a binary classification problem, and we can use a basic machine learning algorithm from scikit-learn to build a predictive model.

We generate synthetic data where the number of hours studied is used to determine whether a student passes the exam. We add a bit of random noise for realism.

The data is visualized with a scatter plot.

The data is reshaped to fit the scikit-learn model requirements.

The data is split into training and testing sets.

A logistic regression model is created and trained on the training set.

Predictions are made on the test set.

The model's accuracy and confusion matrix are printed to evaluate its performance.